Report – Parallel Prefix Scan

Gerald Fattah

CS378H – Assignment 1

September 15th, 2025

Step 1 - Parallel Implementation

Algorithm Approach:

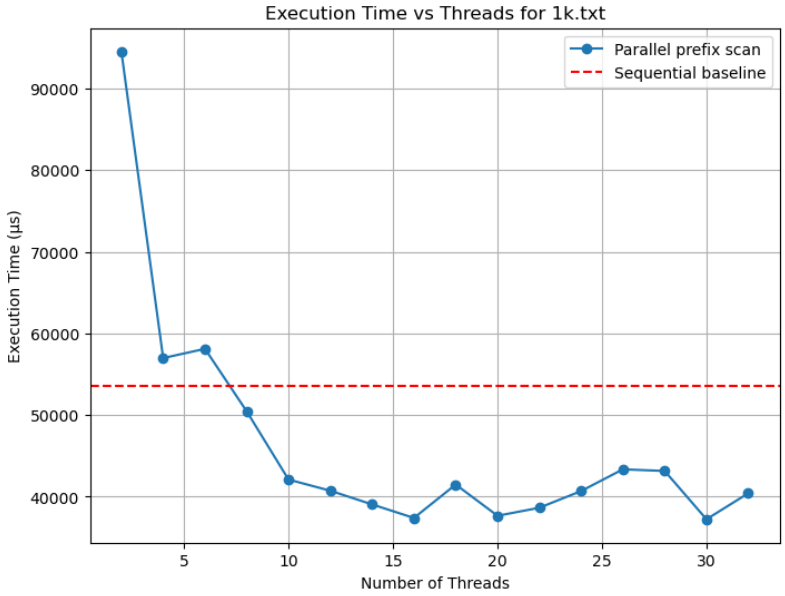
I implemented a parallel prefix scan using the Blelloch scan algorithm (upsweep/downsweep), which is work-efficient since it does O(n) operations, the same as the sequential version. Each thread works on a portion of the array at each level of the tree, and barriers ensure all threads stay synchronized before moving on.

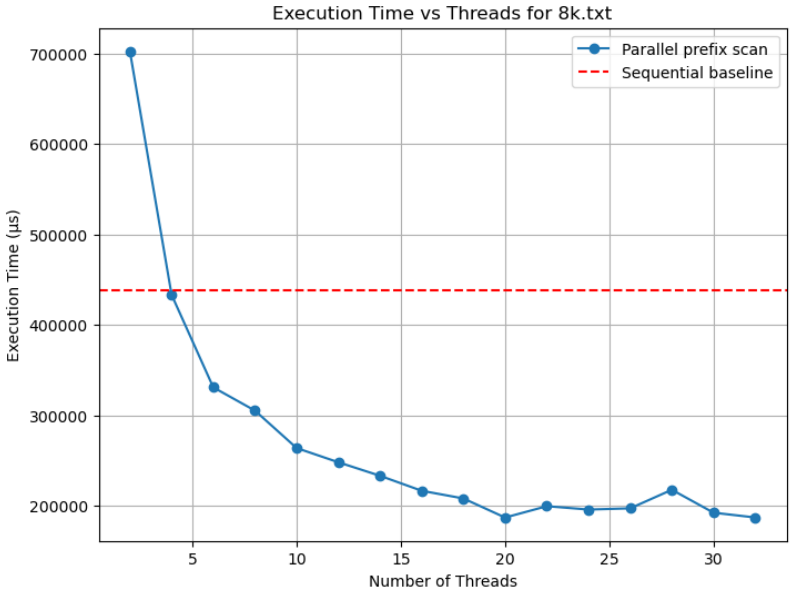
If the number of threads is set to zero, the program runs a straightforward sequential scan. Otherwise, it spawns worker threads that execute the parallel scan function, which handles buffer setup, the upsweep, downsweep, and writing the final results. Execution time is measured with std::chrono as implemented in the skeleton code provided.

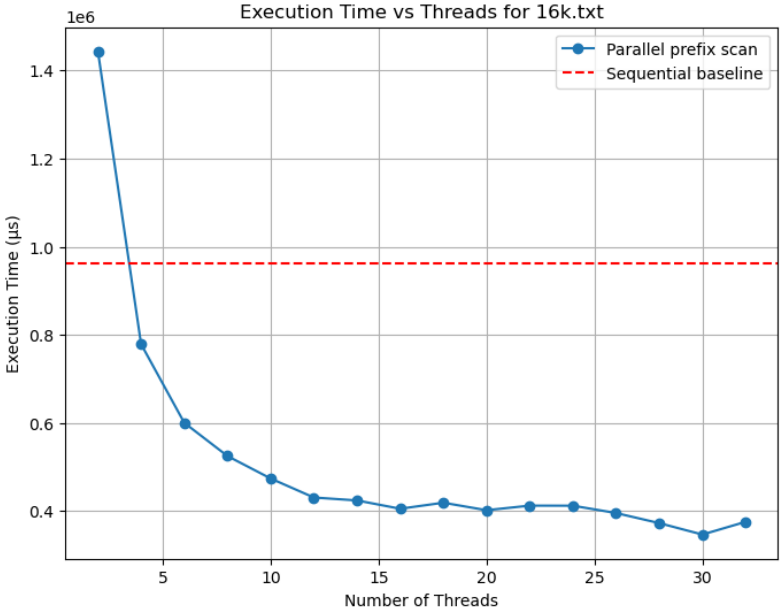
For this step, I ran the program with an operator loop count of 100000 (-l 100000) and varied the number of threads from 2 to 32 in steps of 2. The provided test files (1k, 8k, 16k) were used as input, and the sequential baseline was measured by running with -n 0.

The results showed a clear pattern. For very small inputs, such as 64 elements, the sequential scan was always faster because the parallel version never caught up—the cost of thread management and barrier synchronization outweighed the tiny amount of work. For larger inputs, the parallel implementation started slower when using just a few threads (2–4), but as more threads were added, performance improved and surpassed the sequential baseline, particularly in the range of 8–16 threads. However, as the thread count increased further toward 24–32, the performance flattened or even worsened due to barrier contention, scheduling overhead, and memory bandwidth limits. These results can be seen in figure 1 below.

From these experiments, I concluded that while both sequential and parallel scans have the same O(n) complexity, the parallel version only provides benefits when the input size is large enough to offset synchronization overhead. For small inputs, the sequential implementation is clearly better. On larger inputs, the parallel scan can achieve good speedup, but only up to the point where hardware bottlenecks like core availability and memory bandwidth limit further gains.







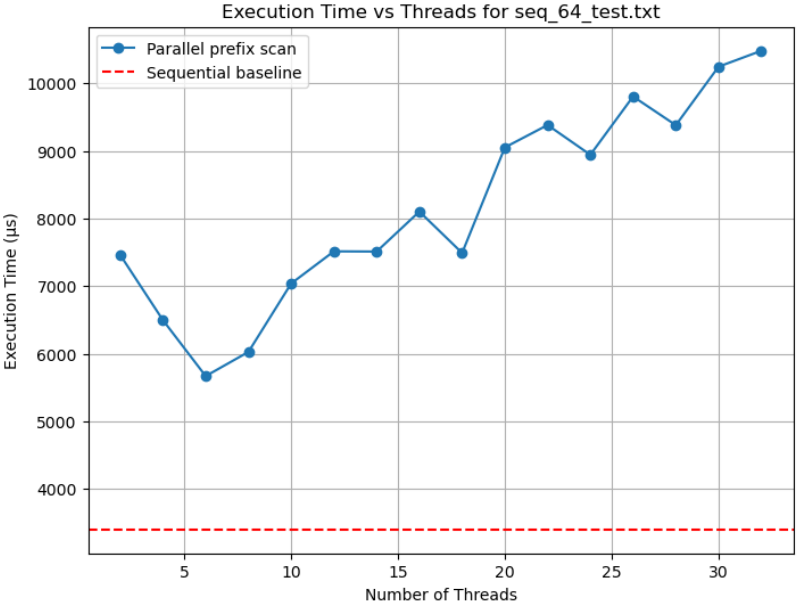


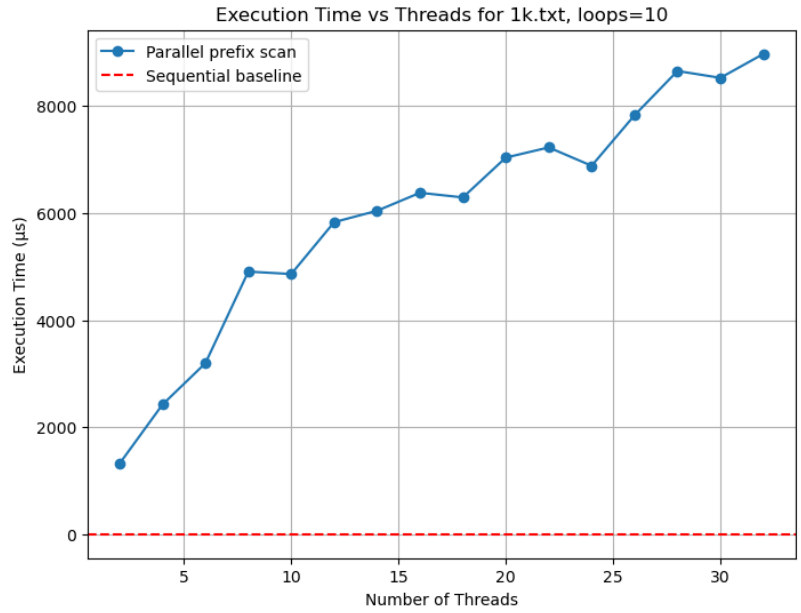
Figure 1 - Parallel Implementation incrementing thread count

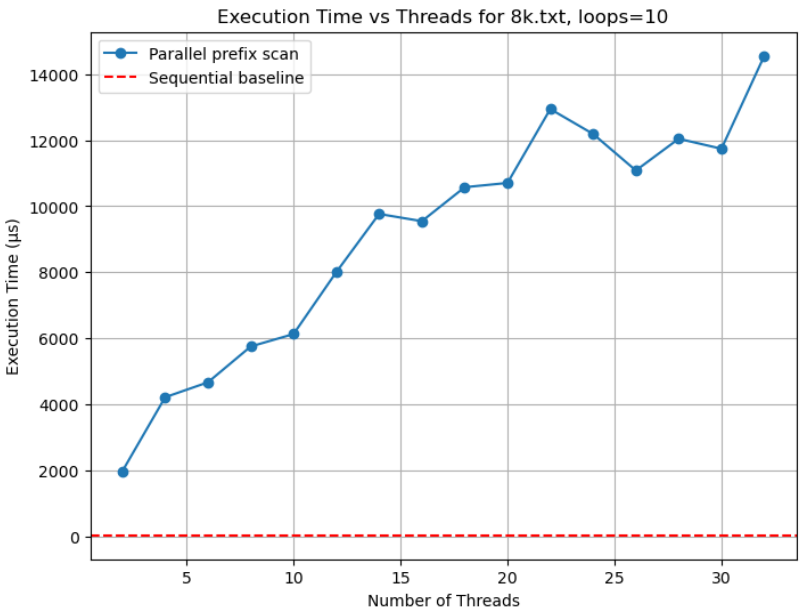
Step 2 – Playing with Operators

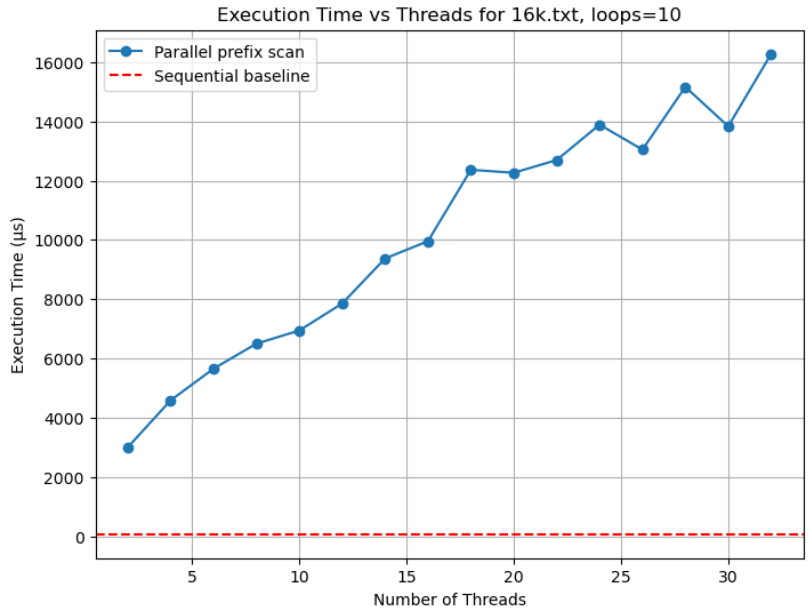
For this part, I kept the same parallel prefix scan implementation but reduced the operator workload by setting the loop count to 10 (-l 10). The experiment was run over the same input set, varying threads from 2 to 32 in increments of 2, and the sequential version (-n 0) was measured as a baseline. I then repeated the runs with increasing loop counts (-l 100, -l 1000, -l 10000, etc.) to find the point where sequential and parallel performance were similar.

When the operator does very little work (-l 10), the sequential implementation was consistently faster. This is expected, since the parallel version spends most of its time synchronizing threads at barriers while the sequential scan runs straight through. At this scale, barrier overhead dominates any benefit from parallelism. As the loop count increases, the operator cost grows and parallel execution becomes more competitive. Around -l 1000, the sequential version was still faster across multiple input sizes (notably the 1k, 8k, and 16k element cases), showing that synchronization was still a limiting factor. Once the loop count was raised further, the crossover points appeared: at about 3000 loops, the 16k test case became faster in parallel; at 5000 loops, the 8k case crossed over; and at 32000 loops, the 1k case finally favored parallel execution. Interestingly, when the operator workload was pushed all the way to 1 million loops, the sequential 64-element case once again became faster, since the data size was so small that the barrier cost outweighed any benefit from splitting the heavy computation. These results highlight how both operator complexity and input size influence whether parallelization is effective.

The main takeaway is that the relative cost of the operator determines whether parallelization pays off. With very cheap operators, the overhead of splitting work and synchronizing multiple threads outweighs the benefits. With heavier operators, the work per element dominates and parallelization can provide speedup. In more general terms, the loop count -l is just a proxy for operator cost: if an operator is expensive, parallel execution is worthwhile; if it's cheap, the sequential scan is better. The inflection point happens where the two costs balance, and that threshold shifts depending on the input size and system.







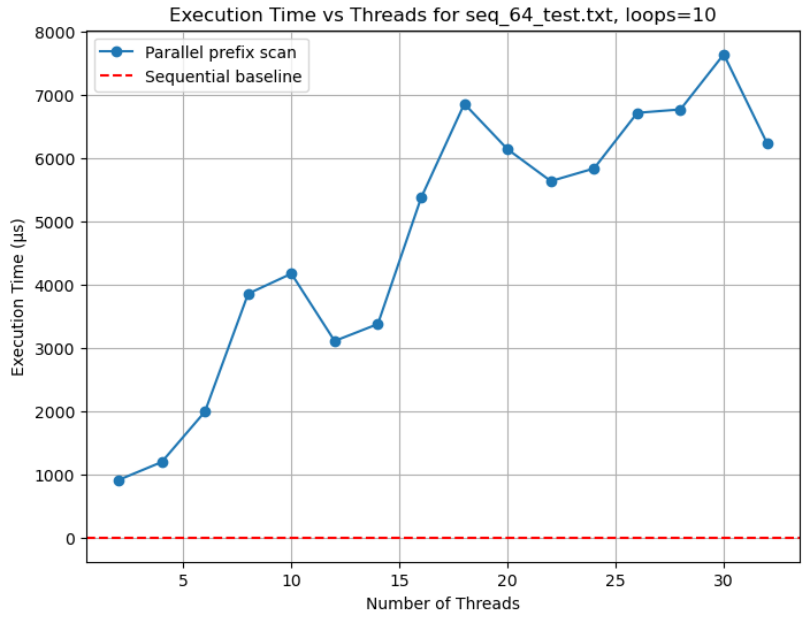
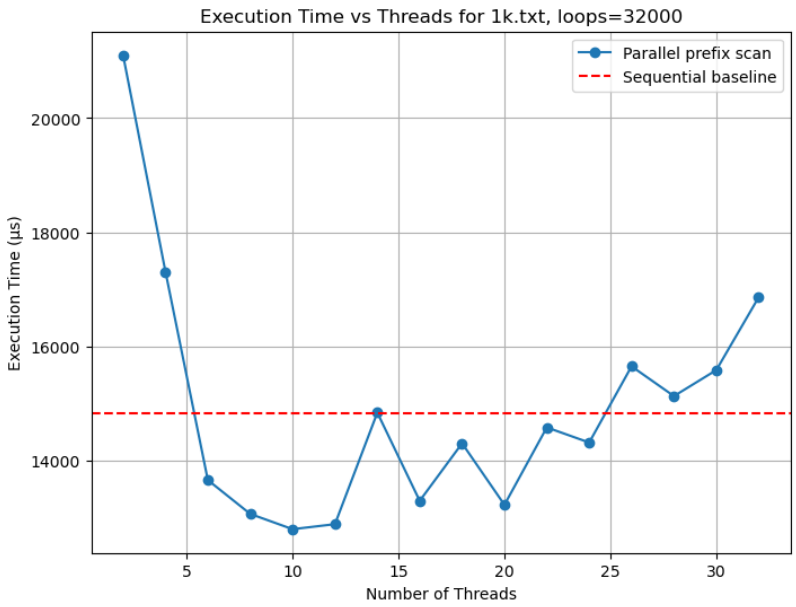
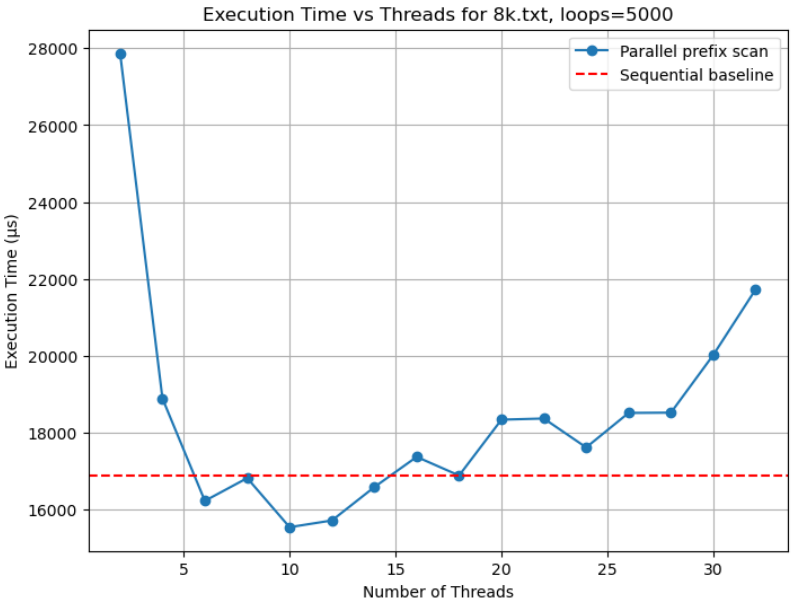
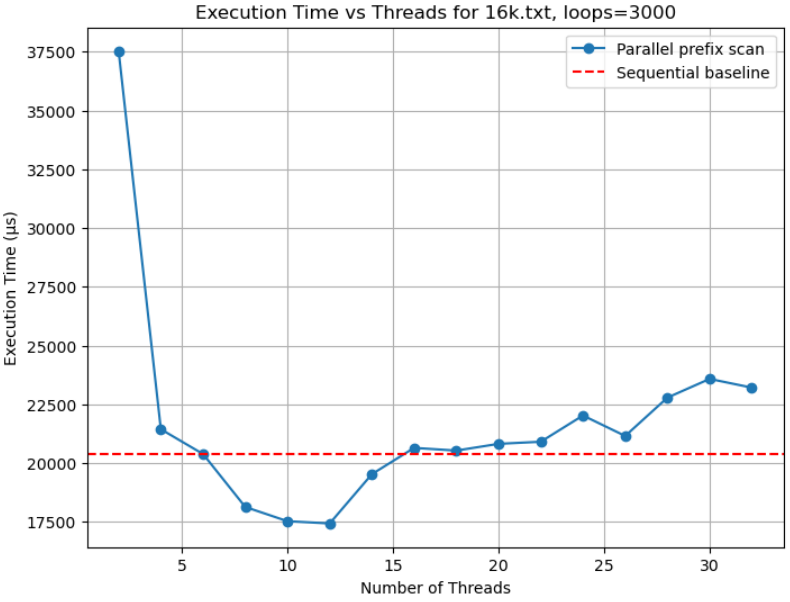


Figure 2 – 10 loops







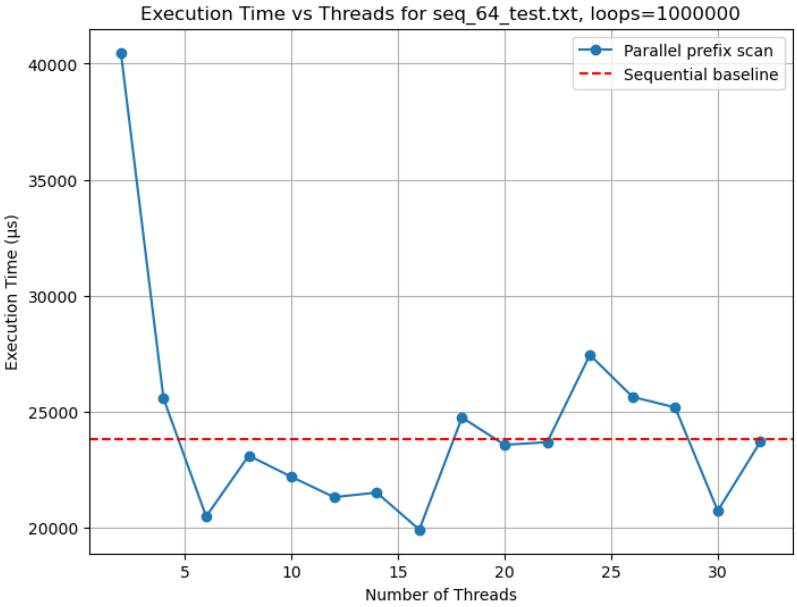


Figure 3 – Point of incidence for all test cases (changing loops)

Step 3 – Barrier Implentation

For this step, I replaced the use of pthread_barrier_t with my own custom re-entrant barrier, my_barrier_t. My implementation is based on atomics rather than condition variables, making it closer to a spinlock design. The barrier keeps track of how many threads have arrived (count), the number of threads required to release (trip_count), and the current generation/level. Each thread atomically increments count when it reaches the barrier. The last thread to arrive resets count, advances the generation, and allows all other threads to proceed. Waiting threads spin until the generation changes, which ensures re-entrancy across multiple barrier usages.

Comparison to pthread_barrier_t:
The main difference is that pthread_barrier_t is implemented using the kernel's synchronization primitives (mutexes and condition variables), which tend to block threads when waiting. My barrier instead uses busy-waiting with atomics, so threads spin in user space until the barrier is released. This makes my barrier lightweight in terms of system calls, but potentially more CPU-intensive when threads wait for long periods.

Scenarios where each is better:

- My barrier performs better when the number of threads is small, the critical sections between barriers are short, and cores are available. In these cases, the spin-waiting overhead is minimal and avoids the cost of kernel context switches.
- pthread_barrier_t is better when there are many threads or when work between barriers is large enough that spinning wastes CPU cycles. In such cases, blocking threads reduces wasted computation and allows the OS to schedule other work.

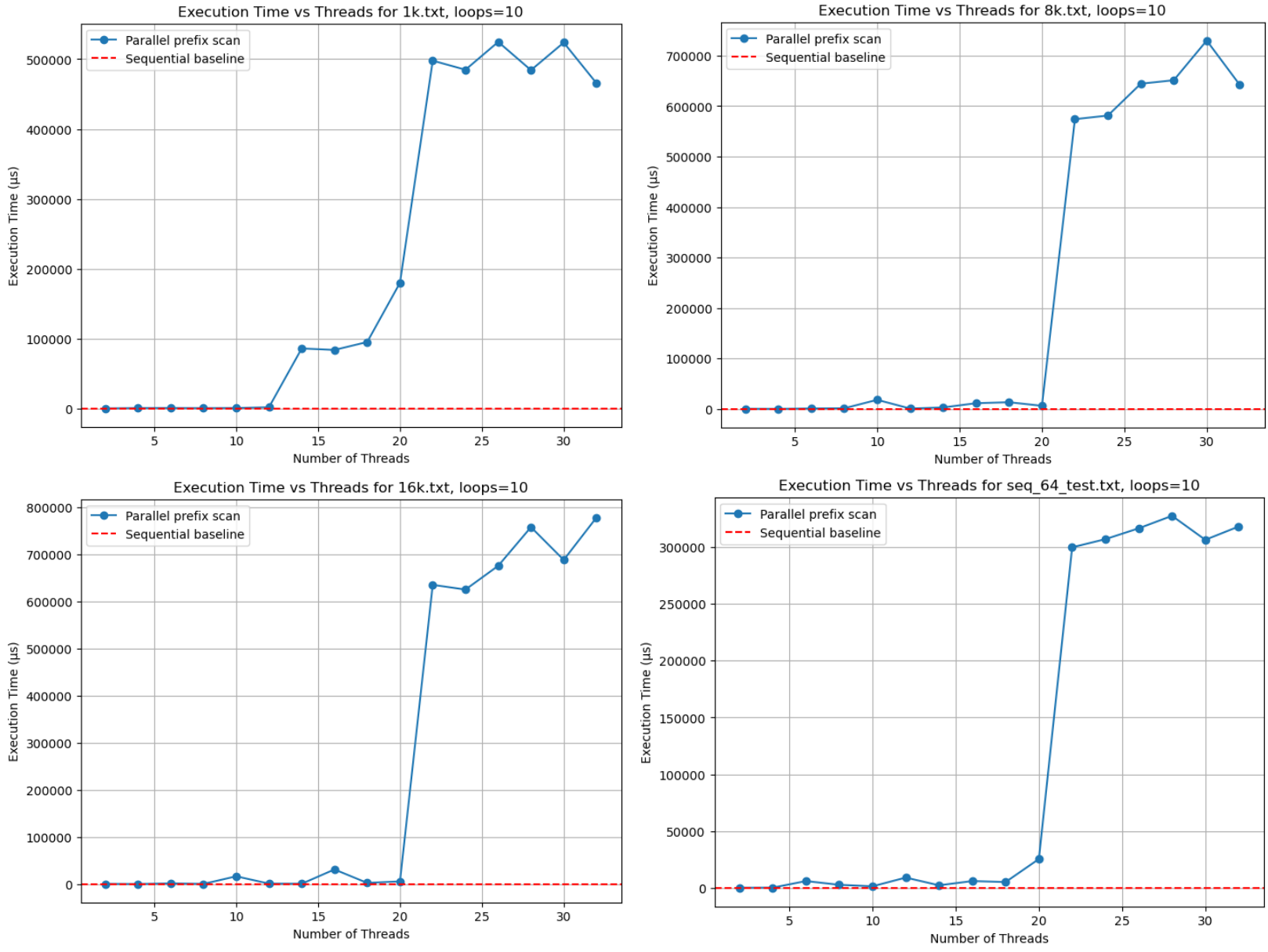
Pathological cases:

- For my implementation, the worst case is when threads are imbalanced (e.g., one thread lags far behind). All other threads will spin-wait, wasting CPU cycles. This can cause severe performance degradation, especially at high thread counts.
- For pthread_barrier_t, the pathological case comes from heavy barrier reuse with short computations in between. The overhead of context switches dominates, making synchronization disproportionately expensive.

Performance results:
I re-ran the prefix scan experiments using my custom spin-lock barrier with loop counts of 10, 100, and 10,000. At low thread counts, performance was close to the sequential baseline, though synchronization overhead kept it from being faster. As the thread count increased, the parallel implementation initially scaled reasonably well, but once the

number of threads exceeded available hardware cores (around 20–22 on my system), execution times spiked sharply. This collapse was caused by the busy-wait design of the custom barrier: idle threads consumed CPU cycles while waiting, leading to contention and wasted work. Compared to pthread_barrier_t, the custom barrier consistently underperformed, with the gap most visible at higher thread counts.

.



.

Figure 4 – 10 Loops Spin-Lock Barrier

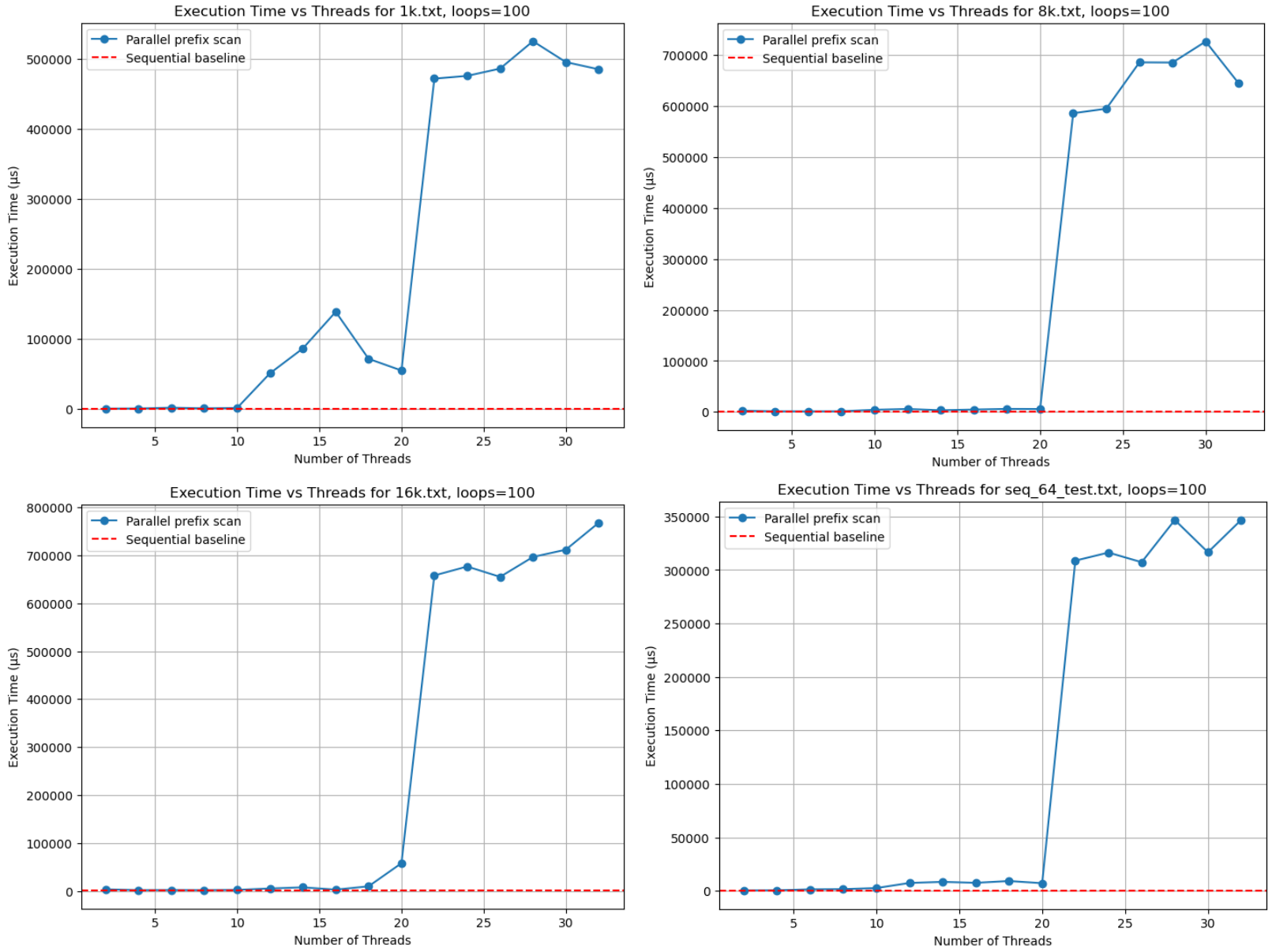
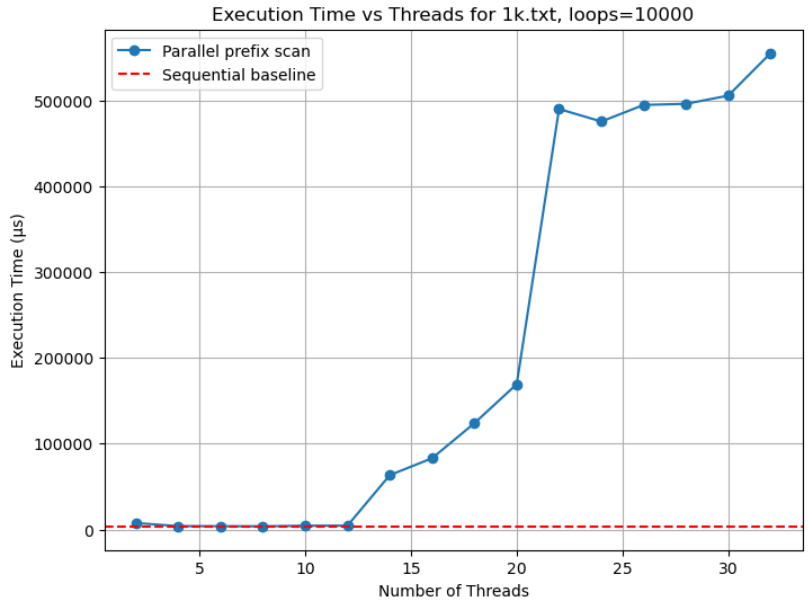
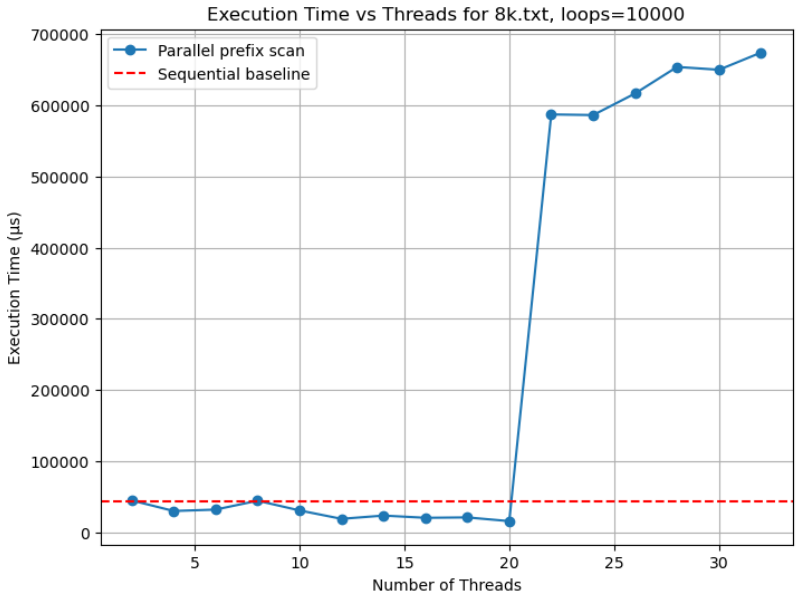
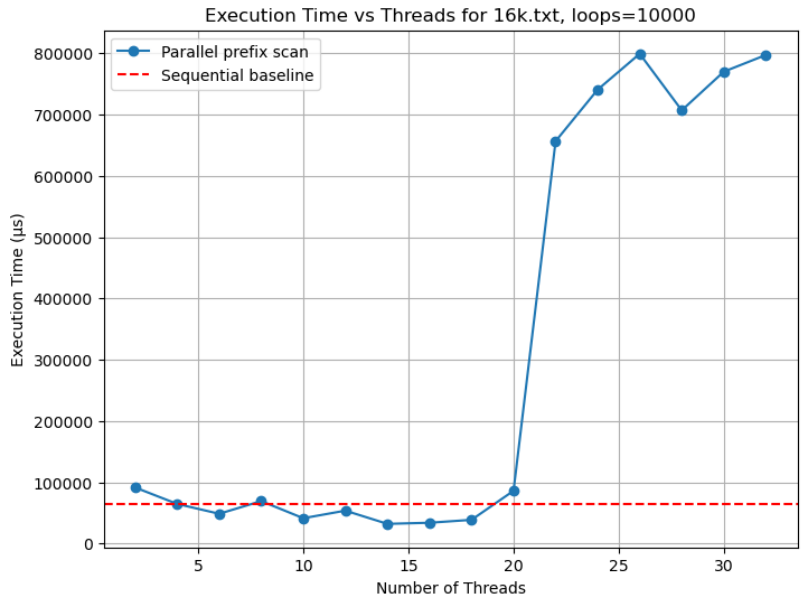


Figure 5 – 100 Loops Spin-Lock Barrier







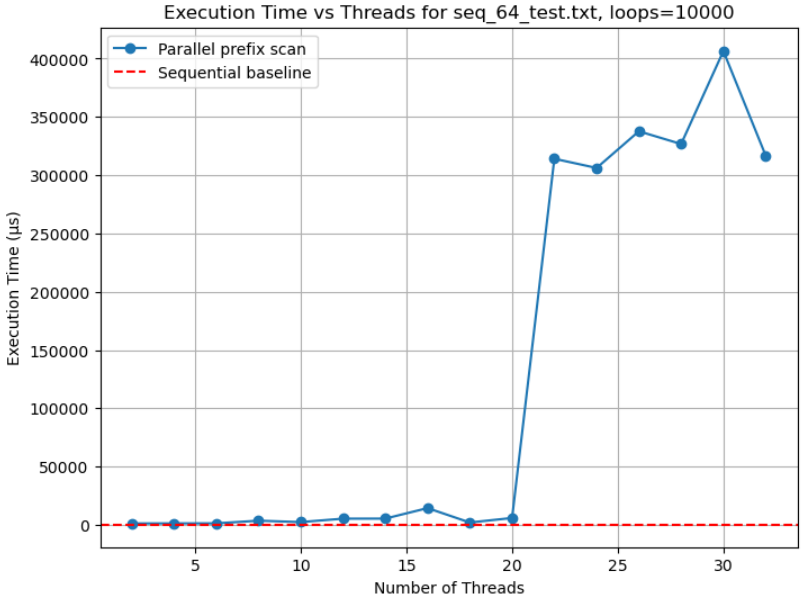


Figure 6 – 10000 Loops Spin-Lock Barrier

Takeaways:

These results highlight how much barrier design affects performance: when the amount of work per thread is small, synchronization costs can dominate. The pthread_barrier_t version tends to look worse on very lightweight workloads (small inputs or very low loop counts) because the cost of entering the kernel outweighs the actual computation. In

contrast, a spin barrier can look slightly better in tightly controlled cases—like low-latency settings with threads pinned to separate cores—since it avoids the system call. But the tradeoff shows up quickly: once the thread count exceeds the number of available cores, the spin barrier suffers from contention, with threads burning cycles while waiting. It also performs poorly when workloads are uneven or long-running, since stragglers hold everyone up while others just spin. My custom barrier is simpler and avoids kernel switches, but wastes resources in those uneven cases. By comparison, pthread_barrier_t blocks idle threads and uses the CPU more efficiently. In the end, neither approach achieves perfect speedup—the scan still has O(n) work—but the real-world performance is shaped heavily by the cost of synchronization.